

# BendRT: A Parallel Runtime for CPUs and GPUs

Victor Taelin  
Higher Order Company  
Rio de Janeiro, Brazil  
taelin@higherorderco.com

AI DISCLOSURE. Bend and its runtime were designed by the human author. This paper was written with the assistance of Claude (Fable 5.1, Anthropic), from the author’s code and design notes, and reviewed by the author.

## Abstract

Bend is a pure functional language with an affine dependent type system. This paper describes BendRT, its runtime. The compiler emits one C file per program; that file runs the same code on one thread, on CPU threads, and on the GPU through Metal or CUDA, over one heap, with no garbage collector and no user-written kernels. Three ideas carry the design. Affinity comes from the checker: a value has one owner, so a match frees the node it consumed, a forked task owns its arguments, and sharing exists only where the compiler placed a counted share or a borrow. The evaluator is flat: every function is a case tree inside one worklist function with no C call stack, and every call site reads two ways, sequential and parallel, so one text serves every executor. Tasks live in a *cube*, a fixed 128 by 128 grid of rings that forks along its rows in a grow phase and drains along its columns in a work phase, with no shared queue, no lock and no work stealing. The price is a contract: the program’s forks must split their work evenly. On an Apple M4 Max the sequential build runs within 0.8 to 1.5 times the time of hand-written C, sixteen threads give 9 to 12 times over one, and the integrated GPU up to 67 times.

## 1 Introduction

Bend’s type system, developed in a companion paper [13], is affine: a live value is consumed at most once unless its binder is marked `+`, and a `+` binder forms only at the kind `Data`, which excludes functions, arrays and handles. The theory paper argues that this wall makes the language consistent. This paper argues that it also makes the language fast: the runtime gets for free what runtimes for functional languages usually pay most for. A garbage collector exists because nobody knows when a value’s last owner leaves; under affinity the one owner is the match that consumes it, so deallocation is compiled code. Work stealing exists because tasks are cheap to make and hard to place; under affinity a task owns its arguments, so a fork moves memory and never shares it, and the only cross-thread protocol is delivering an answer into a join. A GPU wants flat memory, no recursion and no runtime; under affinity the evaluator needs no collector, and with one discipline on calls no C stack either, so the C file is also the shader.

&lt;

used freely, but only at kind `Data`, so no closure, array or file handle is ever shared. A match consumes its scrutinee, always a parameter or a field of one, never a computed value. A def body is flattened at parse into a *case tree*, lambdas and constructor matches over leaves, and validation annotates every node with its type; erased binders (`-`), types and proofs never reach the emitted code. One law governs what follows: a compiler may drop a copy the source spelled, never add one.

### 3 Compilation

#### 3.1 Tail, Cut, Fork

The runtime has no C call stack for user code, so every call must sit where a jump can replace it. The first pass rewrites each reachable body until every call has one of three shapes and all else is call-free expression code. A *tail* is a saturated call in return position. A *cut* is one call whose result is named; its continuation is minted as a sequential definition `f$k`. A *fork* is a let of two or more calls; its continuation is minted as a joiner `f$j`. Lambdas lift into definitions `f$c` over their captures, so a closure is a definition applied to all but one of its arguments.

Two whole-program analyses then decide the memory traffic. *Share inference* finds the constructor types that wear reference counts: a type is *hot* when some binder of it is used more than once, when a hot type's live field reaches it, or when it is boxed in an array; everything else is built and consumed through plain stores and loads. *Borrow inference* finds the arguments a callee only reads: each live argument of a datatype starts borrowed and flips to owned on any escape, to a fixpoint. A borrowed argument is lent raw and read in place, so a fold over a reused tree costs no count traffic. A small fork-free callee is inlined by running the checker's own evaluator on the call; the rest is left to the emitter.

#### 3.2 The Segment

Each definition becomes one *segment* of the worklist function

counter, so an allocation is a pop or a bump, a free is two stores, and neither crosses a thread. Exhaustion is fail-stop: a numbered error in the header, never a hang.

## 4.2 Ownership at Run Time

Generated code moves values. Three operations cover the cases where it cannot. *Take* consumes a value at a match: read the fields, free the node. *Keep* gives a value a second owner and is emitted exactly at a binder’s second use: the first keep mints a *redirect*, one word holding the node’s address and a count, and turns the local into a pointer at it; the count lives there, never on the node, so an unshared value carries no count anywhere. *Drop* releases an owner: a packed term costs nothing, a counted pointer decrements and only the last one collects, a sole owner collects at once. Take respects sharing: at count one it collapses the redirect and owns the node; above one it copies the fields out and releases its pointer. Every decrement is a release and whoever sees zero acquires first, so a field read never races a free. Only hot types pay any of this, and they seal every stored field with a count at build time.

Drop is the runtime’s only collector: an iterative walk that threads its worklist through the nodes being freed, so it allocates nothing and runs from any code on either processor. There is no tracing, no epoch and no deep copy anywhere: a `+` variable is emitted at every use, and each extra use is the share or the borrow the compiler placed.

## 4.3 Arrays

In the source, `Array<T>` is a perfect binary tree walked by index bits. The backends run it as one flat block: packed 32-bit cells for an array of machine words, one owned term per word otherwise. A read is an indexed load, a write an indexed store, and a read of a boxed cell shares it. Affinity makes the store sound: `Array<T>` is `Type`, never `Data`, so an array has one owner, nobody can observe the mutation, and a copy is explicit. A match on the tree takes both halves in place, and joining two adjacent halves back is free.

back into the row: a fork reply deals its children round-robin across the row's rings, and a runnable reply, a completed continuation or a spawned marked call, is pushed back whole. Tasks whose segment can never fork are skipped: they cannot widen the frontier, and they wait for work. A row stops growing when every ring in it has work or a sweep grew nothing.

*Work* runs at saturation. Every lane drains its column ring, down to a snapshot of the put counter taken at the turn's start, in the sequential world: forks run as consecutive calls on the value stack and cuts as frames, and most of a program's time is spent here, in straight-line code. A join completed by the lane's last delivery runs at once, in the parallel world, so the forks it meets are dealt into the cube through the global cursor and counted toward the next round's frontier. This is the bulk-synchronous rhythm [16]: bursts of exponential fork, then deep sequential focus, then the next wave.

The host pool is up to 128 threads, which claim rows by one fetch-add and meet at one barrier per turn; it opens at a program's first fork. The GPU runs a grow as a dispatch of 128 groups (seeded by one group while  $f < 128$ ) and a work as one thread per lane; between dispatches the host only reads  $f$ .

#### 6.4 Why Nothing Contends

Count what crosses a lane. A push is one fetch-add and two stores. A delivery is one release fetch-sub on the join's own node. A page claim is one fetch-add on the global cursor, once per quantum. That is all: no shared deque, no lock on the task path, no migration, no rebalance, and no task is ever re-read to be placed elsewhere. Every phase is a closed step, which is what lets the same phase bodies run as persistent host threads and as device dispatches.

The price is written into the language. Work stealing [2, 4] repairs an unequal split at run time; the cube does not. If a program forks unequal parts, lanes idle at the end of a work turn, and the runtime declines to correct that: balance is the program's job. The idioms are teachable: fork equal halves of the data or the index space, sequence full-width phases instead of forking phases against each other, keep light work out of forks

| bench             | 1 thread | 16 threads | GPU    | C twin |
|-------------------|----------|------------|--------|--------|
| tree bitonic sort | 6.18 s   | 0.62 s     | 0.43 s | 5.48 s |
| game of life      | 5.46 s   | 0.48 s     | 0.08 s | 6.82 s |
| k-means           | 2.26 s   | 0.26 s     | 0.18 s | 2.03 s |
| mandelbrot        | 4.78 s   | 0.40 s     | 0.08 s | 3.79 s |
| tree matmul       | 3.50 s   | 0.31 s     | 0.22 s | 2.89 s |
| merkle tree       | 4.30 s   | 0.39 s     | 0.08 s | 3.48 s |
| n-body            | 4.89 s   | 0.43 s     | 0.08 s | 4.99 s |
| n-queens          | 5.51 s   | 0.46 s     | 1.24 s | 4.61 s |
| tree radix sort   | 3.89 s   | 0.36 s     | 0.25 s | 2.67 s |
| raytrace          | 6.15 s   | 0.53 s     | 0.21 s | 4.67 s |
| symbolic regr.    | 3.31 s   | 0.29 s     | 0.54 s | 2.22 s |
| terrain           | 2.81 s   | 0.25 s     | 0.13 s | 2.19 s |

**Table 1.** The pinned suite, `bench/runtime/` in the Bend repository: seconds on one Apple M4 Max (pin of 2026-08-31, commit `64fc4b7`) for one thread, sixteen threads, the integrated GPU through Metal, and the hand-written C twin, one C function per Bend definition. Each cell is a warm run then a timed run on an idle

- [9] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. In *High Integrity Language Technology (HILT)*, 2014. ACM, 103–104.
- [10] Eric Mohr, David A. Kranz, and Robert H. Halstead. 1990. Lazy Task Creation: A Technique for Increasing the Granularity of Parallel Programs. In *LISP and Functional Programming (LFP)*, 1990. ACM, 185–197.
- [11] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. 2021. Perceus: Garbage Free Reference Counting with Reuse. In *Programming Language Design and Implementation (PLDI)*, 2021. ACM, 96–111.
- [12] Victor Taelin. 2024. *HVM2: A Parallel Evaluator for Interaction Combinators*. Retrieved from <https://github.com/HigherOrderCO/HVM2/blob/main/paper/HVM2.pdf>
- [13] Victor Taelin. 2026. *BendTT: An Affine Dependent Type Theory*.
- [14] Mads Tofte and Jean-Pierre Talpin. 1997. Region-Based Memory Management. *Information and Computation* 132, 2 (1997), 109–176.
- [15] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *Implementation and Application of Functional Languages (IFL)*, 2019. ACM.
- [16] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Communications of the ACM* 33, 8 (1990), 103–111.
- [17] Philip Wadler. 1990. Linear Types Can Change the World. In *Programming Concepts and Methods*, 1990. North-Holland, 561–581.